# Statistical Learning and Data Analysis - Exercise 2

SVD and PCA

## Q.1

### Objective

In this question, you will implement the power method to estimate the dominant right singular vector of a matrix $A \in \mathbb{R}^{n \times m}$, where $n > m$. You will explore how the choice of initialization affects convergence and visualize the error across iterations. Your implementation will be compared against the true singular vectors obtained from the SVD.

### (A) Generating a Matrix with a Singular Value Gap

To test the power method, you will construct a matrix with a large singular value gap.

```
import numpy as np
np.random.seed(0)
n, m = 100, 50 # A will be n x m

# Generate U and V from a random matrix SVD
A_rand = np.random.randn(n, m)
U, _, Vt = np.linalg.svd(A_rand, full_matrices=False)

# Define singular values with a gap
s = np.linspace(10, 1, m)
s[1:] *= 0.1 # Compress all except the top singular value
S = np.diag(s)

# Construct matrix A
A = U @ S @ Vt
```

Note: We construct $A \in \mathbb{R}^{n \times m}$ using its singular value decomposition:

$$A = U \Sigma V^T.$$

Here, $U$ and $V$ are obtained from the SVD of a random matrix, and we manually define $\Sigma$ to have a large gap between the first and remaining singular values. Try to explain why this gap is useful for the power method.

Plot the singular values of $A$.

### (B) Implementing the Power Method

Implement the power method on $A^T A \in \mathbb{R}^{m \times m}$ to estimate the top right singular vector.

1. Write a function `power_method(A, v0, num_iter)` which:
   - Starts with an initial vector $v_0 \in \mathbb{R}^m$.

- Iteratively computes
$$v_{k+1} = A^T A v_k,$$
and normalizes the result.
- Returns the iterates $v_k$ at each step.

2. Briefly explain what we expect from this algorithm when the initial vector has a nonzero component in the direction of the top right singular vector.

## (C) Initializations

Generate two vectors that will be used for initialization:

1. A random unit vector $v_{\text{rand}} \sim N(0, I)$, normalized to have Euclidean norm 1.
2. A vector $v_{\text{orth}}$ that is **orthogonal to the leading right singular vector** $v_1$.

A convenient way to do this is to compute the SVD of $A$ and take the second right singular vector:
$$v_{\text{orth}} = v_2.$$
Since the right singular vectors are orthonormal, this gives
$$\langle v_{\text{orth}}, v_1 \rangle = 0$$
up to numerical precision.

Alternatively, you may start from a random vector $z$ and project out the component in the direction of $v_1$:
$$v_{\text{orth}} = z - \langle z, v_1 \rangle v_1, \qquad v_{\text{orth}} \leftarrow \frac{v_{\text{orth}}}{\|v_{\text{orth}}\|_2}.$$

3. Verify numerically that
$$|\langle v_{\text{orth}}, v_1 \rangle|$$
is approximately machine precision.
4. Plot the entries of $v_{\text{rand}}$ and $v_{\text{orth}}$ on the same figure.

## (D) Measuring Convergence

Run the power method using both initializations. Use enough iterations to see the long-term behavior of the exactly orthogonal initialization, for example `num_iter = 100` or `num_iter = 200`.

1. For each iterate $v_k$, compute the alignment with the true top singular vector $v_1$ using
$$\text{error}_k = 1 - |\langle v_k, v_1 \rangle|.$$
Plot this error as a function of iteration $k$ for both initializations. A logarithmic scale for the $y$-axis may be helpful.
2. For the orthogonal initialization, also compute the alignment with the second singular vector $v_2$:
$$1 - |\langle v_k, v_2 \rangle|.$$
Explain whether the iterates first behave as if they are converging to $v_2$.
3. At each iteration $k$, compute the Rayleigh quotient
$$\rho_k = \|A v_k\|_2.$$
Plot $\rho_k$ as a function of $k$, and add horizontal reference lines at $\sigma_1$ and $\sigma_2$.

4. Explain the behavior you observe. In exact arithmetic, if $v_0$ is exactly orthogonal to $v_1$, then the power method applied to $A^TA$ would never create a component in the $v_1$ direction. Therefore, the iteration would converge to the dominant singular direction inside the orthogonal subspace, typically $v_2$.
5. In floating-point arithmetic, however, tiny round-off errors are introduced during the matrix-vector multiplications and normalizations. Once even a very small component in the $v_1$ direction appears, it is amplified by the power method. Since the eigenvalues of $A^TA$ are $\sigma_j^2$, the relative amplification of the $v_1$ component compared to the $v_2$ component is approximately

$$\left(\frac{\sigma_1^2}{\sigma_2^2}\right)^k.$$

Therefore, the orthogonal initialization may first appear to approach $\sigma_2$, but after sufficiently many iterations it should begin to converge to the true leading direction $v_1$ and to the top singular value $\sigma_1$.

## (E) Discussion

Summarize the difference between the random initialization and the orthogonal initialization. In particular, explain why the orthogonal initialization is a useful numerical experiment: it shows that the power method is not only a theoretical algorithm, but also depends on finite-precision numerical behavior.

# Q.2

## Objective

In this question, you will use Principal Component Analysis (PCA) to explore the structure of facial image data. You will visualize the principal components, also known as eigenfaces, reconstruct images using different numbers of PCs, and interpret what information the leading components capture.

## Dataset: Olivetti Faces

We use the Olivetti Faces dataset from the `sklearn.datasets` module. It consists of 400 grayscale face images of size $64 \times 64$, each flattened into a 1D vector of length 4096. There are 40 unique individuals, each with 10 different images.

```
from sklearn.datasets import fetch_olivetti_faces
faces = fetch_olivetti_faces()
X = faces.data # shape (400, 4096)
images = faces.images # shape (400, 64, 64)
```

## (A) PCA Decomposition

1. Standardize the data by centering it, i.e. subtract the mean image.
2. Apply PCA and compute the first $K = 100$ components.
3. Plot the cumulative explained variance ratio as a function of $K$.

## (B) Visualizing Eigenfaces

1. Visualize the first 10 principal components as $64 \times 64$ grayscale images, as subplots.
2. These are called eigenfaces. They represent dominant patterns of variation across faces. Discuss: What types of features appear in the first few components? For example, lighting, face orientation, and coarse facial structure.

## (C) Face Reconstruction

1. Choose a few test images from the dataset, for example 2-3 images.
2. Reconstruct them using different numbers of components, for example $K = 5, 10, 25, 50, 100$.
3. For each reconstruction, compute and plot the reconstruction error.
4. Visualize the original vs. reconstructed images side by side, again using subplots.

# Q.3

## Objective

In this question, you will connect the PCA reconstruction from Q.2 to the rank-$K$ approximation theorem. The goal is to understand why reconstructing images by keeping only the largest singular values is mathematically justified, and not just a numerical trick.

## (A) Programming: Truncated SVD Reconstruction

Let $X_c$ be the centered data matrix from Q.2, after subtracting the mean face. Compute the singular value decomposition

$$X_c = U\Sigma V^T.$$

For each $K \in \{5, 10, 25, 50, 100\}$, construct the truncated rank-$K$ approximation. If

$$U_K = [u_1, \ldots, u_K], \qquad \Sigma_K = \mathrm{diag}(\sigma_1, \ldots, \sigma_K), \qquad V_K = [v_1, \ldots, v_K],$$

then

$$X_K = U_K \Sigma_K V_K^T.$$

Equivalently, if your code returns `U, s, Vt`, you may compute

```
X_K = U[:, :K] @ np.diag(s[:K]) @ Vt[:K, :]
```

1. Add the mean face back to $X_K$ and reconstruct the same test images used in Q.2(C).
2. Visualize the original images and their rank-$K$ reconstructions side by side.
3. Compute and plot the Frobenius reconstruction error

   $$\|X_c - X_K\|_F$$

   as a function of $K$.
4. Compute and plot the operator norm reconstruction error

   $$\|X_c - X_K\|_2$$

   as a function of $K$.
5. Verify numerically that

   $$\|X_c - X_K\|_2 = \sigma_{K+1},$$

   where $\sigma_{K+1}$ is the first singular value that was discarded.
6. Compare this to the Frobenius norm identity discussed in class:

   $$\|X_c - X_K\|_F^2 = \sum_{j>K} \sigma_j^2.$$

## (B) Theory: Rank-$K$ Approximation in Operator Norm

In class, we discussed the Frobenius norm version of the rank-$K$ approximation theorem. Now prove the corresponding result for the operator norm.

Let

$$A = \sum_{j=1}^{r} \sigma_j u_j v_j^T, \qquad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r > 0,$$

and define

$$A_K = \sum_{j=1}^{K} \sigma_j u_j v_j^T.$$

Prove that

$$\min_{\text{rank}(B)\leq K} \|A - B\|_2 = \|A - A_K\|_2 = \sigma_{K+1}.$$

**Hint.** For the upper bound, compute $\|A - A_K\|_2$ directly. For the lower bound, let $B$ be any matrix with $\text{rank}(B) \leq K$. Consider the subspace

$$S = \text{span}\{v_1, \ldots, v_{K+1}\}.$$

Since $\dim(S) = K + 1$ but $\text{rank}(B) \leq K$, there exists a nonzero vector $x \in S$ such that $Bx = 0$. Normalize it so that $\|x\|_2 = 1$, and show that

$$\|(A - B)x\|_2 = \|Ax\|_2 \geq \sigma_{K+1}.$$

Conclude that $\|A - B\|_2 \geq \sigma_{K+1}$ for every rank-$K$ matrix $B$.

## (C) Interpretation

Explain in words why this theorem justifies the PCA reconstruction procedure. In particular, explain why discarding the small singular values gives the best possible rank-$K$ approximation in operator norm, and how this relates to the face reconstructions you observed in Q.2.